

Submission Worksheet

Submission Data

Course: IT114-003-F2025

Assignment: IT114 Module 3 User Input Challenges

Student: Alanor S. (ass89)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 10/17/2025 7:22:00 AM

Updated: 10/17/2025 8:40:31 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-module-3-user-input-challenges/grading/ass89>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-003-F2025/it114-module-3-user-input-challenges/view/ass89>

Instructions

- Overview Link: <https://youtu.be/iowHMCKuj5o>
- 1. Ensure you read all instructions and objectives before starting.
- 2. Create a new branch from main called M3-Homework
 - 1. `git checkout main` (ensure proper starting branch)
 - 2. `git pull origin main` (ensure history is up to date)
 - 3. `git checkout -b M3-Homework` (create and switch to branch)
- 3. Copy the template code from here: [GitHub Repository - M3 Homework](#)
 - It includes CommandLineCalculator, SlashCommandHandler, MadLibsGenerator, a BaseClass and a stories folder with 5 stories (used for MadLibsGenerator). Put all into an M3 folder or similar (adjust package reference at the top if you chose a different folder name).
 - Immediately record to history
 - `git add .`
 - `git commit -m "adding M3 HW baseline files"`
 - `git push origin M3-Homework`
 - Create a Pull Request from M3-Homework to main and keep it open
- 4. Fill out the below worksheet
 - Each Problem requires the following as you work
 - Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
 - Update the ucid variable
 - Code solution (add/commit periodically as needed)
- 5. Once finished, click "Submit and Export"
- 6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin M3-Homework`
 - 4. On Github merge the pull request from M3-Homework to main

7. Upload the same PDF- to Canvas
8. Sync Local
 1. git checkout main
 2. git pull origin main

Section #1: (3 pts.) Challenge 1 - Command Line Calculator (Add/sub)

Progress: 100%

≡ Task #1 (3 pts.) - Edit the `main` method to solve the requirements

Progress: 100%

Details:

- Don't adjust the give code unless noted
- Challenge 1: Accept two numbers and an operator as command-line arguments (+ and -)
- Challenge 2: Allow integer and floating-point numbers
 - Ensure correct decimal places in output based on input (e.g., $0.1 + 0.2 \rightarrow 1$ decimal place)
- Display an error for invalid inputs or unsupported operators
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (Capture 5 variations of tests)

```
1  # CommandLineCalculator.java
2  package my;
3
4  /**
5   * Challenge 1: Command-Line Calculator
6   *
7   * - Accept two numbers and an operator as command line arguments.
8   * - Support addition (+) and subtraction (-)
9   * - Allow integer and floating point numbers
10  * - Ensure correct decimal places in output based on input (e.g., 0.1 + 0.2 = 1 decimal place)
11  * - Display an error for invalid inputs or unsupported operators
12  * - Capture 5 variations of tests
13  */
14
15  public class CommandLineCalculator extends BaseClass {
16      private static String ucid = "uc12345"; // ucid change to your ucid
17
18      /**
19       * public static void main(String[] args) {
20       *     // Implement your solution here. Implement a calculator using command line arguments.
21       * }
22       */
23      if (args.length != 3) {
24          System.out.println("Usage: java my.CommandLineCalculator <number> <operator> <number>");
25          return;
26      }
27      double result = 0.0;
28      try {
29          System.out.println("Calculator result: ");
30          // Implement your solution here
31          double num1 = Double.parseDouble(args[0]);
32          double num2 = Double.parseDouble(args[1]);
33          char operator = args[2].charAt(0);
34      } catch (Exception e) {
35          System.out.println("Invalid input. Please use only numbers and operators (+, -).");
36      }
37  }
```

code part 1

```
1  # CommandLineCalculator.java
2  package my;
3
4  /**
5   * Challenge 1: Command-Line Calculator
6   *
7   * - Accept two numbers and an operator as command line arguments.
8   * - Support addition (+) and subtraction (-)
9   * - Allow integer and floating point numbers
10  * - Ensure correct decimal places in output based on input (e.g., 0.1 + 0.2 = 1 decimal place)
11  * - Display an error for invalid inputs or unsupported operators
12  * - Capture 5 variations of tests
13  */
14
15  public class CommandLineCalculator extends BaseClass {
16      private static String ucid = "uc12345"; // ucid change to your ucid
17
18      /**
19       * public static void main(String[] args) {
20       *     // Implement your solution here. Implement a calculator using command line arguments.
21       * }
22       */
23      if (args.length != 3) {
24          System.out.println("Usage: java my.CommandLineCalculator <number> <operator> <number>");
25          return;
26      }
27      double result = 0.0;
28      try {
29          System.out.println("Calculator result: ");
30          // Implement your solution here
31          double num1 = Double.parseDouble(args[0]);
32          double num2 = Double.parseDouble(args[1]);
33          char operator = args[2].charAt(0);
34      } catch (Exception e) {
35          System.out.println("Invalid input. Please use only numbers and operators (+, -).");
36      }
37  }
```




Section #2: (3 pts.) Challenge 2 - Slash Command Handler

Progress: 100%

≡ Task #1 (3 pts.) - Edit the `main` method to solve the requirements

Progress: 100%

Details:

- Don't adjust the give code unless noted
- Challenge 1: Accept user input as slash commands (Commands are case-insensitive)
 - `/greet <name>` → Prints "Hello, <name>!"
 - `/roll <num>d<sides>` → Roll <num> dice with <sides> and returns a
 - `/echo <message>` → Prints the message back
 - `/quit` → Exits the program
- Challenge 2: Print an error for unrecognized commands
- Challenge 3: Print errors for invalid command formats (when applicable)
- Add code to solve the problem (add/commit as needed)

🖼 Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with uid/date comment)
2. Full output of executing the program (Capture 3 variations of each command except `/quit`)

```

1  // Sample code for the challenge
2  package com.example.challenge2;
3  import java.util.Scanner;
4  public class Main {
5      public static void main(String[] args) {
6          Scanner scanner = new Scanner(System.in);
7          // Loop until user enters 'quit'
8          while (true) {
9              String input = scanner.nextLine().trim();
10             if (input.isEmpty()) {
11                 continue;
12             }
13             // Split the input by space
14             String[] parts = input.split(" ");
15             // Check for 'quit' command
16             if (parts[0].equalsIgnoreCase("quit")) {
17                 System.out.println("Exiting the program.");
18                 break;
19             }
20             // Check for 'greet' command
21             if (parts[0].equalsIgnoreCase("greet")) {
22                 if (parts.length < 2) {
23                     System.out.println("Error: /greet requires a name.");
24                     continue;
25                 }
26                 String name = parts[1];
27                 System.out.println("Hello, " + name + "!");
28             }
29             // Check for 'roll' command
30             if (parts[0].equalsIgnoreCase("roll")) {
31                 if (parts.length < 3) {
32                     System.out.println("Error: /roll requires number of dice and sides.");
33                     continue;
34                 }
35                 int numDice = Integer.parseInt(parts[1]);
36                 int sides = Integer.parseInt(parts[2]);
37                 // Simulate rolling dice
38                 int sum = 0;
39                 for (int i = 0; i < numDice; i++) {
40                     sum += (int) (Math.random() * sides) + 1;
41                 }
42                 System.out.println("Rolling " + numDice + "d" + sides + ": " + sum);
43             }
44             // Check for 'echo' command
45             if (parts[0].equalsIgnoreCase("echo")) {
46                 if (parts.length < 2) {
47                     System.out.println("Error: /echo requires a message.");
48                     continue;
49                 }
50                 String message = String.join(" ", parts[1]);
51                 System.out.println(message);
52             }
53             // Default: unrecognized command
54             else {
55                 System.out.println("Error: Unrecognized command.");
56             }
57         }
58     }
59 }

```

code pt1

code pt2

code pt3

code pt4

output



Progress: 100%

Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1

<https://github.com/AICSS442/asa89-IT1141003M3-Homework/M3/SlashCommandHandler.java>



URL

<https://github.com/AICSS442/asa>



Saved: 10/17/2025 8:28:02 AM

Part 3:

Progress: 100%

Details:

Briefly explain **how** the code solves the challenges (note: this isn't the same as **what** the code does)

Your Response:

This code builds a command parser that interprets and executes user input dynamically. It continuously reads user input, interprets commands starting with a slash and uses string parsing and condition checks to decide what to do. I made sure to convert the input to lowercase off the bat to make the commands case-insensitive and easy to validate later down the line. For each recognized command, it performs input validation, processes the command logic, and outputs the correct response. If an input is invalid or improperly formatted, it triggers specific error messages. Since it's in a loop, it keeps prompting the user for input until they use the /quit command.



Saved: 10/17/2025 8:28:02 AM

Section #3: (3 pts.) Challenge 3 - Mad Libs Generator

Progress: 100%

Task #1 (3 pts.) - Edit the `main` method to solve the challenges

Progress: 100%

Details:

- Don't adjust the given code unless noted
- Ensure you have the **stories** folder with the 5 stories
- Challenge 1: Load a **random** story from the "stories" folder
- Challenge 2: Extract **each line** into a collection (i.e., ArrayList)
- Challenge 3: Prompts user for each placeholder (i.e., **<adjective>**)
 - Any word the user types is acceptable, no need to verify if it matches the placeholder type
 - Any placeholder with underscores should display with spaces instead

- **Challenge 4: Replace placeholders with user input (assign back to original slot in collection)**
- **Add code to solve the problem (add/commit as needed)**

Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment)
2. Full output of executing the program (Capture the process for at least 2 stories)

```

1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     int data;
6     Node* next;
7 };
8
9 Node* head = NULL;
10 Node* tail = NULL;
11
12 void insertAtEnd(int data) {
13     Node* newNode = new Node(data);
14     if (head == NULL) {
15         head = tail = newNode;
16     } else {
17         tail->next = newNode;
18         tail = newNode;
19     }
20 }
21
22 void display() {
23     Node* temp = head;
24     while (temp != NULL) {
25         cout << temp->data << " ";
26         temp = temp->next;
27     }
28     cout << endl;
29 }
30
31 int main() {
32     int n;
33     cout << "Enter number of nodes: ";
34     cin >> n;
35     for (int i = 1; i <= n; i++) {
36         insertAtEnd(i);
37     }
38     display();
39     return 0;
40 }

```

code pt1

[illegible]

code pt2

code pt3

[illegible]

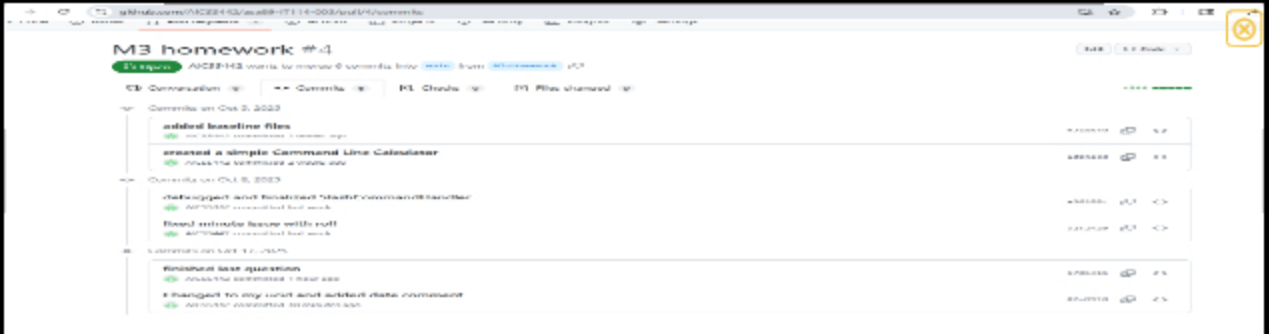
Progress: 100%

Part 1:

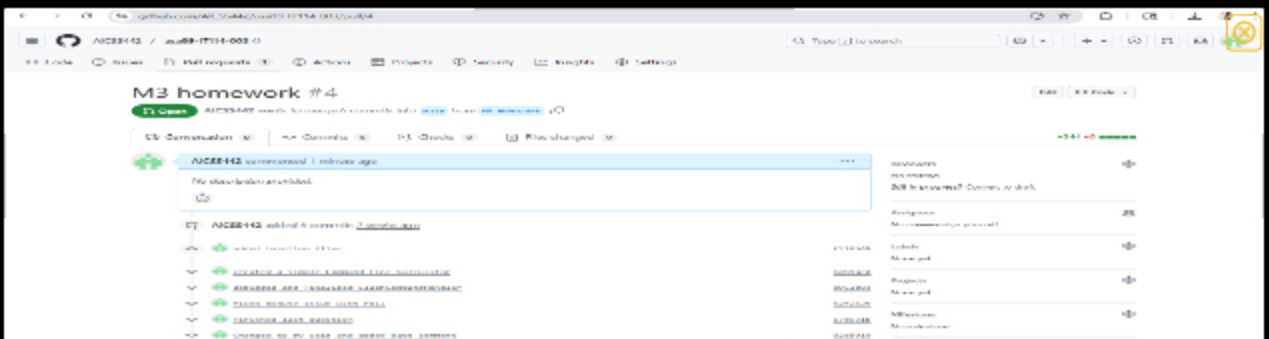
Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present



Commit History



Pull request page

Saved: 10/17/2025 8:06:41 AM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/AICSS442/asa89-IT1114003/>



URL

<https://github.com/AICSS442/asa>

Saved: 10/17/2025 8:06:41 AM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- **Note:** The duration isn't relevant for the grade and the visual graphs aren't necessary

Projects • asa89-IT114-003



44 mins over the Last 7 Days in asa89-IT114-003 under all branches.

not accurate b/c it doesn't account for the time I spent early last week on this assignment without being a premium member



Saved: 10/17/2025 8:09:52 AM

☰ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to access a folder, read text files, and randomly select one to process. I created a File object pointing to the folder that had the stories, which tells java where to look for the story files. Then I got all the files inside of that folder by using the `.listFiles()` method. I then used the Random class to choose a story at random and used Scanner class to read its contents into an ArrayList



Saved: 10/17/2025 8:35:44 AM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I found the first question to be the simplest part of the assignment. I had the most exposure to that type of logic and I only had to worry about 2 operators.



Saved: 10/17/2025 8:39:16 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I think number 2 was the most difficult question because there was a lot more error cases to handle. This made the code a little more complicated. Specifically, the dice problem since it had the most error cases to handle.



Saved: 10/17/2025 8:40:31 AM